

P-Gaussian: Provenance-Based Gaussian Distribution for Detecting Intrusion Behavior Variants Using High Efficient and Real Time Memory Databases

Yulai Xie^{1b}, *Member, IEEE*, Yafeng Wu, Dan Feng, *Member, IEEE*, and Darrell Long^{2b}, *Fellow, IEEE*

Abstract—It is increasingly important and a big challenge to detect intrusion behavior variants in today's world. Previous host-based intrusion detection methods typically explore the sequence of system calls or unix shell commands to detect the intrusion behavior. This article abstracts the detection of intrusion behavior variants as the comparison between different sequences when the sequence order or length transforms. To overcome the impact of sequence transformation on the detection accuracy, we propose P-Gaussian, a provenance-based Gaussian distribution detection scheme which comprises two key design features: (1) it utilizes provenance to describe and identify intrusion behavior variants, and eliminates the impact of sequence order transformation on the detection accuracy. (2) it adopts Gaussian distribution principle to accurately compute the similarity between intrusion behavior and its variant, and eliminates the impact of intrusion behavior sequence length increase on the detection accuracy. To improve the detection performance, P-Gaussian employs a Redis memory database with multiple Redis instances and multiple threads to enable the parallelism of provenance processing in multi-core environments. It also classifies hot and cold provenance to provide high-efficient long-term forensic analysis. Experimental results on widely-used real world applications demonstrate the performance and efficiency of our system.

Index Terms—Provenance, intrusion behavior variant, Gaussian distribution, real-time, memory database

1 INTRODUCTION

TODAY'S enterprises are facing a wide variety of intrusion threats such as Advanced Persistent Threat (APT). Usually, these attacks are stealthy and can change their behaviors to evade detection. Typically, a malicious attacker can change his attack behavior constantly to make the intrusion detection software fail to track his footprints. A worm can add some irrelevant operations to counter the removal of antivirus software. These intrusion behavior variants can inflict severe damage to the computers in the world. For instance, a new version of Zeus virus infected a large number of Facebook users in 2013 [1]. In 2017, variants of Wannacry virus infected 3600 computers per hour [2].

Typical intrusion behavior variant detection methods in host-based intrusion detection system fall into two categories: static analysis and dynamic analysis. The former focuses on

static code analysis without executing them. The typical methods include binary analysis [3] and mining the frequency of strings [4]. These methods have a limited detection performance in the case of code transformations [5]. The latter concentrates on the exploitation and exploring of the global information or control flow graph [6], [7], [8], [9]. The disadvantage lies in the difficulty of analysis and big time overhead generated in a restricted environment [10].

An invasion process can be characterized by a number of system call sequences of a certain length [11]. By comparing the sequences extracted from the intrusion behavior with the sequences in the rule database, whether the attack has occurred can be judged. The typical sequence similarity calculation methods include the Hamming distance [12], LCS algorithm [13], and GST algorithm [14]. However, these methods are seriously affected by two factors: sequence order transformation and sequence length increase, which are very common when an intrusion process changes its behavior to a variant. For sequence order transformation, as the sequence to be detected changes into a different sequence, the traditional system-call based intrusion detection methods (e.g., the Hamming distance [12], Hidden Markov Models [15]) may fail to identify this variation if the rule database does not contain the corresponding sequence variant. Alternatively, as algorithms such as LCS depend on the sequence length to compute the sequence similarity, the detection result will be not accurate if a sequence variant has added a lot of unrelated items and makes its length greatly increase.

- Y. Xie and Y. Wu are with the School of Cyber Science and Engineering, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: {ylxie, yafengwu}@hust.edu.cn.
- D. Feng is with the School of Computer, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: dfeng@hust.edu.cn.
- D. Long is with the Jack Baskin School of Engineering, University of California, Santa Cruz, CA 95064 USA. E-mail: darrell@ucsc.edu.

Manuscript received 27 Nov. 2018; revised 20 Oct. 2019; accepted 12 Dec. 2019. Date of publication 17 Dec. 2019; date of current version 11 Nov. 2021. (Corresponding author: Yulai Xie.)
Digital Object Identifier no. 10.1109/TDSC.2019.2960353

To address the above challenges, we designed P-Gaussian, a provenance-based Gaussian (i.e., normal) distribution scheme to describe and compute the similarity of two sequences. Provenance represents the history or lineage of an object, and has been commonly used in applications, such as experimental documentation, search [16], and security [17]. Compared to the system call, provenance can more accurately and explicitly describe the dependency relationship between intrusion process and the infected files. When the provenance sequence order transforms, e.g., the behavior “intrusion process P first reads file A, and then reads file B” changes into “intrusion process P first reads file B, and then reads file A”, the dependency relationships are still $P \rightarrow A$ and $P \rightarrow B$. So provenance-based method can still work when detecting such an intrusion behavior variant. Alternatively, Gaussian distribution has been used in intrusion detection to filter network traffic data [18] or provide deployment strategy in wireless sensor networks [19], [20]. As the common length m is a random variable for any two sequences, m approximately obeys the Gaussian distribution [21]. Even if one of the sequences is extracted from intrusion behavior variant that has added a lot of unrelated items to increase the sequence length, the common length m between two sequences still does not change. The corresponding similarity probability in Gaussian distribution also does not change. So in this case, unlike the LCS method that is greatly impacted by the sequence length increase, Gaussian distribution scheme can still identify the anomaly sequence in the variants even though the detected sequence length increases.

In addition, unlike existing approaches that use GPU [22] or a simple memory database [23] to improve detection speed, P-Gaussian can improve the parallelism of sequence similarity comparison in big data environments by employing multiple Redis instances with multiple threads in multi-core environments. It also optimizes the intrusion detection performance by speeding up the provenance query. Typically, a provenance object can have multiple ancestors or descendants in a provenance graph. P-Gaussian stores all these dependency information only in one record by using one set to store all these ancestors or descendants. The provenance query can be more efficient when reading them from only one provenance record.

P-Gaussian also enforces forensic analysis by classifying the hot and cold event data and storing them in memory and disk respectively. Further, the provenance data for each event is clustered together in separate physical location to speed up the query. For the cold provenance that does not contain intrusion information and seldomly accessed, P-Gaussian compresses it using the classical compression techniques such as bzip2 [24] to save the storage space.

We develop a prototype system to collect and optimize the provenance of normal and intrusion behavior, generate the provenance rule database to store the frequently accessed intrusion provenance paths, and then judge whether the intrusion has happened by employing provenance based Gaussian distribution scheme to compute the similarity value between a provenance path and the items in the rule database. It calculates the sum of similarity values of all the provenance paths formed in an intrusion based on the weights of these paths in different lengths. Thus, the intrusion behavior

described as a provenance graph can be detected using the Gaussian distribution scheme.

P-Gaussian is similar to existing approaches (e.g., PIDAS [25], Pagoda [23] and FRAP [26]) that also detect intrusions in real time using provenance. The difference is that they are not designed for detecting intrusion behavior variants, while P-Gaussian can detect the variants using Gaussian distribution scheme when intrusion behavior sequence length increases. In addition, P-Gaussian can improve detection speed by providing value aggregation and parallel provenance processing by employing multiple Redis instances with multiple threads. It also speeds up forensic analysis by classifying hot and cold provenance.

The contributions of this paper are as follows:

- We make a detailed analysis of a variety of sequence similarity comparison methods in case of sequence order transformation and sequence length increase.
- We propose P-Gaussian, a provenance-based Gaussian distribution scheme to describe and accurately identify the intrusion behavior and its variant, especially when the intrusion behavior sequence transforms.
- We develop a system prototype that can achieve high efficient and real-time detection performance on big provenance streams by exploiting the parallelism of provenance processing using multiple Redis instances and multi-threading technology in multi-core environments. It also enforces long-term forensic analysis by classifying the hot and cold event data and storing them in memory and disk respectively.
- We evaluate the scheme and the system prototype on various real-world normal and vulnerable applications. The experimental results demonstrate the performance and efficiency of the scheme and system.
- We released the provenance data sets for related research on provenance based intrusion detection and forensic analysis.

We propose setting and motivation in Section 2 and analyze a variety of sequence similarity comparison methods in case of sequence transform in Section 3.2. We elaborate the design of provenance-based Gaussian distribution in Section 4 and its prototype system in Section 5. In Sections 6 and 7, we evaluate the system performance and overhead, and discuss the implementation issues respectively. We summarize the related work in Section 8 and conclude the paper in Section 9.

2 BACKGROUND AND SETTING

We first give an overview of provenance and provenance-based intrusion detection. Then we describe the threat model we use in this paper.

2.1 Provenance and Provenance-Based Intrusion Detection

Provenance records how data come to its current state and can usually be presented as a directed acyclic graph. The nodes in the graph can be objects such as files, processes, sockets or any data sets. Each node can have attributes such as name, id, version number, and port number. The edges

reveal the data or control flow between different objects. The provenance can be collected via a number of provenance systems, including PASS [27], [28], SPADE [29], Story Book [30], ES3 [31], TREC [32], HiFi [33], Eidetic System [34], and CamFlow [35]).

The existing provenance-based intrusion detection systems typically collect the provenance of intruders based on systems such as PASS, SPADE, and CamFlow. PASS generates file-level provenance via intercepting system calls. SPADE further supports distributed provenance capture on multiple platforms. CamFlow was recently proposed to support whole-system provenance capture and take the security into consideration. They can identify intrusions by exploring the anomaly degree of a single provenance path (i.e., PIDAS [25]), the whole provenance graph (i.e., FRAP [26]), or the combination of a single path with the whole graph (i.e., Pagoda [23]). These methods are similarity-based detection methods and are easily impacted when the length of the provenance sequence (i.e., path) changes. For PIDAS, it detects intrusions by computing the anomaly degree of a provenance path. Assume the rule database stores the frequently occurring intrusion behavior sequences, a provenance path generated by an intrusion behavior has been judged as an anomaly. In its intrusion behavior variant, an unrelated edge that does not appear in the rule database is added to this path and the anomaly degree of the provenance path decreases to not exceed the threshold. Thus the intrusion behavior variant will not be identified. For Pagoda, though it judges intrusions by considering both the specific provenance path and the provenance graph, the anomaly degree of the provenance graph will be decreased to below the threshold when a number of normal edges are added to the provenance paths. Especially the clever adversary can make use of this to evade the detection. For FRAP, it transforms each normal provenance graph into a vector by relabeling each provenance nodes during multiple times of iterative process. Then it clusters the similar vectors using k-means clustering method and identifies the isolated vectors that represent anomalous provenance graphs. However, the variant can incorporate a lot of normal edges into its provenance graph, making it similar to the normal provenance graphs. For modern attacks, variants with the transformation of sequences have become prevalent. It is challenging to identify the variants quickly.

In Pagoda [23], a persistent memory database has been used to store provenance and aggregates multiple similar items into one provenance record to maximumly reduce unnecessary I/O during the detection analysis. With the multi-core environment, we would like to enable the multi-threaded access to the memory database to fully exploit the hardware to further improve the detection performance. In addition, current provenance systems [36], [37] usually adopt the provenance compression technology to improve the forensic analysis efficiency. In big data environments where provenance increases significantly, it is necessary to manage provenance in a more efficient way. For instance, APT attacks can lurk in enterprises for at least half a year. During this period, the administrator can make multiple times of forensic analysis or system constructions. Thus some provenance data (e.g., some common header files or

shared library files) can be accessed frequently. Meanwhile, some provenance data may have been never accessed. It is necessary to classify hot and cold provenance to further improve the forensic analysis efficiency.

2.2 Threat Model and Assumptions

The primary threat we guard against in this paper is the intrusion behavior or its variant via exploiting the application or process code vulnerabilities. The typical intrusion behavior can include reading or tampering with the data in the system, or downloading any malware to the system. The variant can add redundant operations, reverse operations, have packers or be encrypted. The provenance system can intercept system calls during this process, generate provenance and identify the anomaly in provenance.

The provenance system cannot track the intrusions that do not go through the syscall interface. Typically, the malicious code attack that causes a memory leak or system corruption cannot be recorded. For instance, exploration of OpenSSL heartbleed vulnerability (CVE-2014-0160) cannot be detected by this approach as the data leak from memory does not produce system call and generate provenance. In addition, the malicious code self-producing is currently not tracked in this scheme.

As provenance collection process is in the kernel, we assume that the trusted computing platform [38], [39] is employed to protect the kernel from attack. There is also a possibility that the provenance store is removed or tampered with by the intruders. We can also employ sophisticated security scheme [40] to prevent the provenance from unauthorized and undetected modification.

3 ANALYSIS OF INTRUSION BEHAVIOR SEQUENCE TRANSFORMATION

The typical intrusion behavior sequence transformation includes two aspects: sequence order transformation and sequence length increase. We analyze a variety of sequence similarity comparison methods in terms of these two aspects.

3.1 Sequence Order Transformation

The sequence order transformation in intrusion behavior variant has a great impact on the system call based intrusion detection method, but has no effect on provenance-based intrusion detection method. Because when the syscall sequence order changes, the sequence becomes a new one, this increases the difficulty of detection. However, it is almost unaffected when the provenance information is utilized in intrusion detection.

For instance, a virus process P first reads files A and B, and then writes file C. The provenance graph and system call sequence are shown in Fig. 1a. After the order of intrusion behavior changes to first writing file C and then reading files A and B, the provenance graph does not change, but the system call sequence becomes different as shown in Fig. 1b. This indicates that the expression of provenance graph is not affected by the intrusion behavior sequence order transformation.

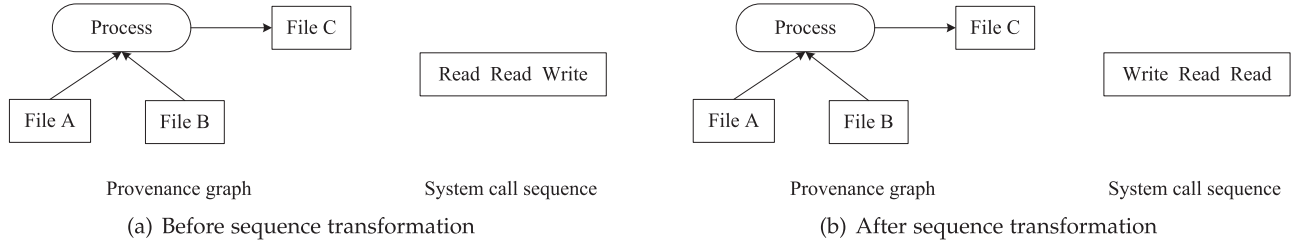


Fig. 1. Provenance graph and system call sequence during sequence transformation. Before the sequence transformation, the virus process P first reads files A and B, and then writes file C. After the sequence transformation, the virus process P first writes file C, and then reads files A and B.

3.2 Sequence Length Increase

As the intrusion behavior detection accuracy is determined by how we judge the similarity between a variant sequence being detected and a sequence in the rule database. Assume the length of the sequence in rule database is n , the length of the variant sequence increases to m , the common length between these two sequences is L , and the similarity value between them is P . We investigate a series of characteristics that a proper sequence similarity method should satisfy as follows.

- P should not be impacted much by the m .
- P is 0 when L equals 0 and P is 1 when L equals n .
- P is not independent for different L .

The first characteristic is a must to detect the variant whose length increases a lot. This excludes the methods that need the length of the variant as input when computing the similarity. Typically, the traditional sequence similarity detection methods (e.g., LCS algorithm) find the longest common subsequence of two sequences and then calculate the similar extent of two sequences as follows.

Let two sequences be S and Q : $S = \{S_1, S_2, \dots, S_m\}$, $Q = \{Q_1, Q_2, \dots, Q_n\}$. The longest common subsequence of S and Q is: $R = \{(S_i, Q_j) | S_i = Q_j, S_i \in S (1 \leq i \leq m), Q_j \in Q (1 \leq j \leq n)\}$. Then the S and Q similarity calculation formula is defined as

$$P(S, Q) = \frac{2 \times |R|}{|S| + |Q|}; \quad (1)$$

The maximum value of $P(S, Q)$ is 1, and can be achieved just when two sequences are completely the same; its minimum value is 0, which is achieved only when the intersection of sequence S and Q is empty.

However, this approach requires that the length difference between the two sequences cannot be too large. Otherwise, the similarity will not be good. For instance, if the length of an intrusion behavior sequence is 3, and the length of another intrusion behavior sequence variant is 10, even if the variant has three same items as the former, the similarity ratio is only $3/(3 + 10) = 23.1\%$. So in this case, the sequence variant will probably not be detected as abnormal.

As for the second characteristic, we observe two phenomena as follows in the intrusion detection process:

- 1) Even if the sequence to be detected is normal, it will have at least one common item as the sequences in the rule database.
- 2) Even if the sequence to be detected is extracted from intrusion behavior, the sequence will not exactly match the sequence in the rule database. This is

because the intrusion behavior is not static, and there are always a variety of variants. For instance, a malicious attacker can adapt its behavior to evade the detection. A worm or virus can also transform to a variant by adding some unrelated behaviors or reverse its original intrusion steps.

Assume the number of ordinary items that appear in the sequences of a rule database is 30, and the length of the sequence is 10. The possible number of sequence S is A_{30}^{10} . The possible number of sequence Q that has no common item with P is A_{20}^{10} . The possible number of combinations of any two sequences is $A_{30}^{10} \times A_{30}^{10}$. Then the probability of two sequences that have no common item is

$$P_0 = \frac{A_{20}^{10} \times A_{30}^{10}}{A_{30}^{10} \times A_{30}^{10}} = \frac{A_{20}^{10}}{A_{30}^{10}} = 0.0061.$$

The probability of two sequences that are completely the same is

$$P_{10} = \frac{A_{30}^{10}}{A_{30}^{10} \times A_{30}^{10}} = \frac{1}{A_{30}^{10}} \approx 0.$$

It can be observed that, the probabilities of two sequences that exactly match or do not match at all are both very small. Especially, the probability that two sequences are completely the same is close to 0. That means, if a sequence is a random sequence, it is almost impossible to completely match the sequences in the rule database. If a sequence to be detected has been transformed, this will further increase the difficulty of matching.

The third characteristic indicates that the similarity values at different common length have inherent relationships. Specifically, when L ranges from 0 to n , according to the second characteristic, the probabilities that L equals 0 and n are both very small. Thus there must be a biggest probability when L equals m ($0 \leq m \leq n$). Then the probability increases when L increases from 0 to m and decreases when L increases from m to n . This coincides well with the Gaussian distribution.

4 PROVENANCE-BASED GAUSSIAN DISTRIBUTION

This section elaborates P-Gaussian, a provenance-based Gaussian distribution scheme that eliminates the sequence length or order change on the detection accuracy. It contains two steps. First, it uses Gaussian Distribution to calculate the similarity between each provenance path and the sequences in the rule database. Then it judges whether a provenance graph is generated based on the intrusion behavior (or its variant) by calculating the sum of similarity values of all the provenance paths based on their weights in different lengths.

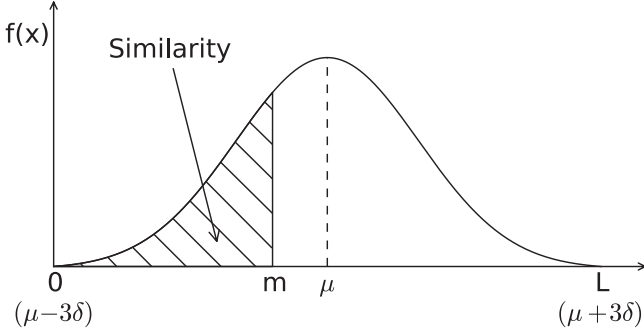


Fig. 2. Gaussian distribution for similar sequence detection.

4.1 Gaussian Distribution Based Sequence Similarity Detection

Suppose there are two sequences: S and Q . S is the sequence in the rule database and its length is L . Q is the sequence to be detected. Assume that the common length m of these two sequences approximately obeys $N(\mu, \sigma^2)$ normal distribution, of which μ is mean, and σ^2 is variance, as shown in Fig. 2. As mentioned in Section 3.2, we define the similarity as the shadow area between the Gaussian distribution curve and the x -axis, and then the similarity of these two sequences can be calculated as follows:

$$\begin{aligned} F(m) &= \Pr p(x < m) \\ &= \int_0^m f(x) \\ &= \int_0^m (1/(\sqrt{2\pi} \times \sigma)) e^{-(x-\mu)^2/(2\sigma^2)}. \end{aligned} \quad (2)$$

In Equation (2), $F(m)$ represents the probability when x is smaller than m , and $f(x)$ is a probability density function of a normal distribution. When $m = 0$, the similarity of two sequences is almost 0; when m becomes bigger, the shadow area becomes bigger, and the similarity value also becomes bigger; when m equals L , the shadow almost covers the area between the curve and x -axis, i.e., the similarity value approximately equals 1.

This probability usually needs to be converted to the standard normal distribution for easier computation. We illustrate it through the same example in Section 3.2. In this case, the original length of the sequence is $L = 3$, and the common length between the variant sequence and the original sequence is $m = 3$, and then we calculate the similarity value as follows:

- 1) Computing μ and σ . As shown in Fig. 2, $\mu = L/2 = 1.5$, $L = (\mu + 3\sigma) - (\mu - 3\sigma) = 6\sigma$, thus $\sigma = L/6 = 0.5$.
- 2) Converting Gaussian distribution to standard normal distribution. Let $Z = (x - \mu)/\sigma \sim N(0, 1)$, then when $x = 3$, $Z = (3 - 1.5)/0.5 = 3$; when $x = 0$, $Z = (0 - 1.5)/0.5 = -3$.
- 3) Computing $F(3)$ as follows:

$$F(3) = \int_0^3 f(x) = \int_{-3}^3 f(Z) = \int_{-3}^3 1/\sqrt{2\pi} e^{-Z^2/2} = 0.9974.$$

This similarity value is much larger than that of LCS. Unlike LCS that is greatly impacted by the sequence length

increase, P-Gaussian is only related to the number of common items (i.e., common length) between different sequences, but has nothing to do with the length of the original sequence to be detected. So this method can cope with the intrusion behavior sequence variant with length increase very easily.

4.2 Provenance Graph Similarity Calculation

For a provenance graph that is generated during an intrusion, P-Gaussian assigns each provenance path (i.e., a provenance sequence) a weight value W according to the length of the path. Let the lengths of the paths in a provenance graph be $\text{len}_1, \text{len}_2, \dots, \text{len}_n$, the corresponding W_i ($1 \leq i \leq n$) is calculated as: $W_i = \frac{\text{len}_i}{\sum_{j=1}^n \text{len}_j}$. P-Gaussian computes the similarity of each path by selecting the maximum similarity value between this provenance path and the sequences in the rule database. Then it multiplies the similarity value of each provenance path by its weight value. At last, P-Gaussian calculates the sum of all the similarity value and compare it to a predefined threshold value T . Assume the similarity values between these provenance paths and the sequences in the rule database are $S_1, S_2, \dots, S_n$, then it computes the total provenance graph similarity P as follows:

$$P = S_1 \times W_1 + S_2 \times W_2 + \dots + S_n \times W_n. \quad (3)$$

If $P > T$, then it judges that the intrusion has occurred.

4.3 Detection in Adversarial Settings

A common assumption in adversarial settings is that the adversary is aware of the detection method used. Suppose the adversary is aware of the provenance graph and wants to construct a new variant. A logical approach here is to apply the analysis presented in 4.1 and determine the maximum common sequence length that would allow him to pass the test in formula (3) and subsequent threshold judgement.

To distinguish the disguise of adversary, one possible method is to use adaptive threshold adjustment. When the variant evades detection by lowering the common length, we can lower the threshold to detect the variant accordingly. But at the same time, in order to avoid false alarms caused by lowering the threshold, we further calculate the similarity between the provenance graph of the variant and the provenance graphs stored in the original rule database through transforming all the provenance graphs into vectors using methods such as FRAP [26] and CamQuery [41]. Only the vector of the provenance graph that is very similar to the rule database is identified as anomalous and thus it is correctly regarded as a variant. In another case, a long common sequence in the variant is likely to have been decomposed into two or more short sequences in order to evade the detection. One possible solution is to mark these small sequences and merge them into one long sequence in the detection process. The merge process is mainly based on two aspects. First, the generation time of these short sequences is the same. Second, as these short sequences originally belong to one long sequence, there exist very strong inherent logical dependency relationships within and between

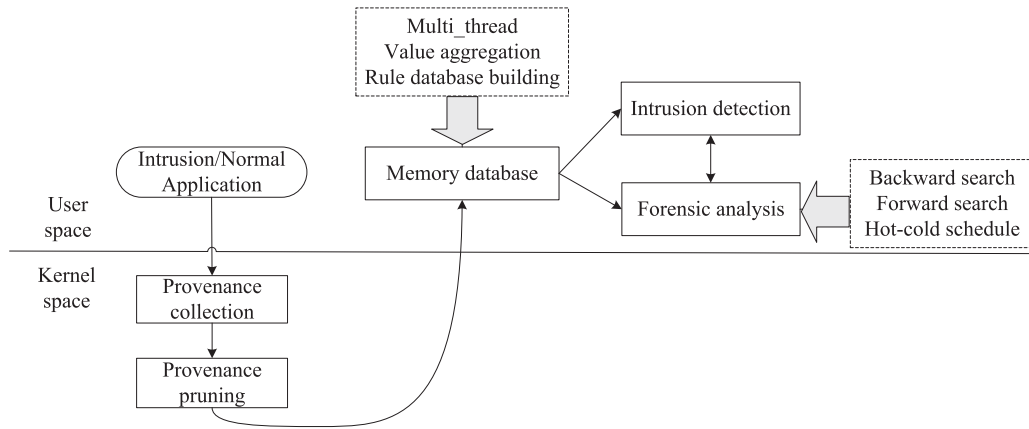


Fig. 3. The prototype system for P-Gaussian.

these sequences. So the variant can be identified correctly via reconstructing the common long sequence accurately.

5 SYSTEM DESIGN AND IMPLEMENTATION

The prototype system for P-Gaussian is shown in Fig. 3. It first intercepts system call information and generate provenance records which can be pruned to store in a memory database. We propose a series of optimization methods such as multi-thread and value aggregation to speedup the access to the memory database. We build rule database in memory database by extracting all the unrepeated provenance sequences from provenance records. Then the intrusion detection process can be performed using the similarity detection algorithm described above. We can also make forensic analysis to find the system vulnerability and intrusion sources by initiating backward and forward queries. Further, the forensic analysis can be optimized by classifying hot and cold provenance and storing them in memory database and disk respectively when the provenance size is large. We describe the details in the following subsections.

5.1 Provenance Collection and Optimization

We collect provenance using the existing PASSv2 [28] framework. Alternative provenance tracking systems can also be HiFi [33], SPADE [29], Story Book [30], and CamFlow [35]. These systems all intercept system call events, generate dependency relationships between files, processes, sockets, and pipes, and convert them to provenance records. We further optimize the provenance information by removing the loop and repetitive records and filtering unrelated files, and then write them to log files. Then, provenance log can be batch-loaded to the database for highly efficient query.

The dependency relationships in provenance mainly include: (1) a process is depended on or depends on another process via commands such as invoke; (2) the relationships between files and processes via read or write command; (3) the SEND and RECEIVE system calls that build the relationships between processes and network sockets.

A provenance graph that describes intrusion process is generally too large and inconvenient to analyze for system administrators. In fact, many nodes and edges for intrusion analysis are not indispensable. For instance, some temporary file nodes can be omitted because they do not store

persistent information. In addition, we also filter some files that are only read but not written (e.g., libraries files or header files) in data provenance information. These files are mainly shared object files and some configuration files. As they do not contain sensitive information (e.g., user data), they are not likely to be the target for the intruder.

In addition, as the pipe has no NAME attribute in provenance information, when we search for paths using NAME as keyword, the path will crack if a pipe is in the path. As each cracked path can be parts of the entire attack, it cannot completely characterize the whole attack behavior. We reconnect the path by adding a NAME attribute for pipe. For example, we can reconnect the path " $A \rightarrow B \rightarrow C \rightarrow pipe$ ", and path " $pipe \rightarrow D \rightarrow E \rightarrow F$ " to a new path " $A \rightarrow B \rightarrow C \rightarrow pipe \rightarrow D \rightarrow E \rightarrow F$ ". In this way, we reconnect the cracked dependence caused by pipes so that the similarity calculation will be more accurate.

We use a series of memory databases such as Redis to store optimized provenance as shown in Table 1. For instance, ProvDB stores all data provenance information. NameDB stores the mapping of object name to its pnode number to speed up the query. Each pnode number uniquely identifies an object. ChildDB and ParentDB store the dependencies between objects and their child nodes or parent nodes. They can be used in path queries via computing transitive closures. RuleDB contains all the frequently occurring dependency relationships that probably reveal the information flow of program behavior. We will elaborate its generation process in the following subsection.

To further optimize the provenance storage and queries, the provenance records with the same key and different values are combined into one provenance record by clustering these values into one set. Typically, a provenance object can have multiple ancestors or descendants. The provenance

TABLE 1
Provenance Database

Database	Key	Values
IdentityDB	pnode No.	attribute information
NameDB	file name	pnode No.
ParentDB	child name	parent name
ChildDB	parent name	child name
RuleDB	child name	parent name

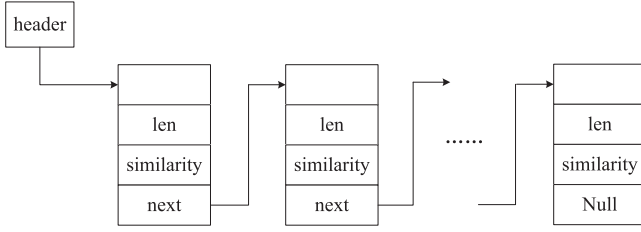


Fig. 4. Provenance path list.

query can be more efficient when reading them from only one provenance record.

5.2 Intrusion Detection

Training Phase. As generating RuleDB can be offline, we use a training set to profile the program behavior by looking for the critical provenance proof. To achieve the completeness of the training set, we have run each program multiple times and stop only the RuleDB is stable. As the behavior of a program can be represented as a provenance graph, we extract the dependency relationships (i.e., provenance edges) from the graph. To guarantee the accuracy of the training set, we only select the relationships that occur above a number of times. We use the dependency relationships as the element of RuleDB as they represent the basic information flow that reveal the intrusion behavior. A sequence in the RuleDB is defined as a provenance path that consists of a series of consecutive dependency relationships. Further, we remove the redundancy in the RuleDB to make it concise and efficient. Typically, P-Gaussian uses an absolute path to describe the name of a file to help locate the damaged files exactly. When an intrusion process access a directory that includes a lot of files, there exist a series of dependency relationships between these files and the process. As these files are in a common directory, the prefixes in their file names are redundancy and can be encoded using an integer.

Detection Phase. For each provenance path in an input provenance graph, the intrusion detection algorithm finds the provenance path in the rule database that has the maximum common length with it, then computes the similarity value using provenance-based Gaussian distribution scheme. As the intrusion behavior is described as a provenance graph, the similarity value between the provenance graph and the rule database is computed based on the similarity value of all the provenance paths and the weight (i.e., path length) for these provenance paths. When the similarity value exceeds a threshold T , the intrusion is judged to have happened.

Specifically, we first search in the ParentDB to find all the head nodes as shown in Algorithm 1. A head node has a lot of children or grandsons, but has no father. Then, in Algorithm 2, we search for all the provenance paths using the head nodes as start points. For each node in the provenance path, we find the sequences in the ruleDB that start from it and then compute the common length between the sequences and the provenance path. The Algorithm 2 first finds the maximum common length between each path in the input provenance graph and the sequences in the ruleDB, and then compute the similarity value using the Gaussian distribution scheme. Then it stores the length of

the path and the similarity value into a path list as shown in Fig. 4. The similarity of the whole provenance graph is computed by traversing the list and calculating the average weight. An alarm will be sounded if the similarity of the whole provenance graph exceeds a predefined threshold.

Algorithm 1. Search for Provenance Head Nodes. Function: FindHead(cname)

Input: ParentDB

Output: list_head /// A list that stores all the head nodes

```

1:  $ret = ParentDB \rightarrow get(cname, pname)$ 
2: if  $ret = DB\_NOTFOUND$  then
3:   add cname to list_head
4:   return
5: while  $ret \neq DB\_NOTFOUND$  do
6:   FindHead(pname)
7:    $ret = ParentDB \rightarrow get(cname, pname, DB\_NEXT\_DUP)$ 
```

Algorithm 2. P-Gaussian: Provenance Based Gaussian Distribution

Input: list_head

Output: alarm

```

1: for each head node p in list_head do
2:   for each path q that starts from p do
3:     for each node j in path q do
4:       Find each path that starts from j in RuleDB, record
         the common length between this path and q
5:     endfor
6:     Select the largest common length m
7:     compute the similarity value  $F(m)$  between path q and
         the sequences in the RuleDB;
8:     add the length (i.e., L) of path q and  $F(m)$  into list_path;
9:   endfor
10: endfor
11: for each  $\langle L, F(m) \rangle$  in list_path do
12:   total_len = total_len + L;
13:   similarity_graph = similarity_graph + L *  $F(m)$ ;
14: endfor
15: similarity_graph = similarity_graph / total_len;
16: if similarity_graph > threshold then
17:   call alarm;
18:   return;
19: endif
```

Detection with Multiple Redis Instances and Multiple Threads. The intrusion detection process can be very efficient in Redis memory database. Originally, Redis uses a single-threaded IO multiplexing model to enable efficient memory database access. This is probably insufficient in big data environments where massive provenance needs to be processed in real time. Typically, detecting and analyzing APT attack usually involves processing and digging multiple suspicious provenance workflows from a big provenance volume generated over a long time [42]. It is a big challenge to do this in real time. We have taken two measures to speed up the system. First, as current prevalent machines support multicore, we have adopted multiple Redis instances to process the intrusion provenance data. Each Redis instance is represented by a host port and is responsible for processing a part of the data. Second, we employ a multi-threaded

program to process a large number of provenance streams in multi-core cases. Each thread is bound to a Redis instance port. Specifically, in the detection process of P-Gaussian, each path in the input provenance graph needs to be compared against the sequences in the RuleDB. The combination method of multiple Redis instance with multi-thread has significantly speeded up this process by maximumly exploiting the parallelism between the multi-cores.

5.3 Forensic Analysis

The detection method also enforces forensic analysis by providing a detection point (typically a suspicious file or process) in an abnormal provenance path when the similarity of the path exceeds a predefined threshold. The forensic analysis typically includes two aspects: backward search and forward search. Backward search aims to find the intrusion source or system vulnerabilities. It usually uses a detection point as a starting point to do path queries in the index databases (ParentDB or ChildDB). The purpose of the forward search is to find all the tampered or leaked files using the intrusion source that we have found. Even in the case that the intrusion behavior variant occurs, the forensic analysis can still help the system administrators to take measures to further enforce the system security (e.g., repairing the software bug).

Forensic Analysis with Hot/cold Data Separation. As Redis takes up significant memory source, it cannot exceed the physical memory. When provenance size exceeds the Redis capacity, part of the provenance has to be loaded to disks. P-Gaussian classifies hot and cold provenance to improve forensic analysis efficiency. First, it starts a daemon program to record the frequently accessed provenance data and store them in memory to reduce disk IO. Typical frequently accessed data includes common used files such as library files and header files that are commonly used in different program executions. Besides, an administrator can first query provenance graphs to identify how intrusion has happened. Then she may recover the damaged files by querying the same provenance graphs again. Caching the hot provenance can speed up the forensic analysis. Second, the cold provenance data for each event is stored together in physical disks to accelerate the query. If the cold provenance has not been accessed for a very long time, it can be further compressed using the classical compression techniques such as bzip2 to save the storage space.

Note that we do not use the Virtual Memory (VM) scheme in Redis for the following reasons. First, the VM has been abandoned after version 2.4 due to its inefficiency. Second, the default VM does not swap keys to disks even a $\langle \text{key}, \text{value} \rangle$ has not been accessed for a long time. In our design, key and value have comparable storage space. Storing the cold keys in Redis in big data environments is not a good solution. Third, the data size for swapping in default VM is a page of 4 KB. Even there is only a few bytes are hot, all the 4 KB data have to be loaded into memory. This is a big waste of space and not efficient to process data in a fine-grained mode.

6 EVALUATION

We first describe the experimental setup including the application and workload, and then we evaluate the P-Gaussian

scheme and system with respect to the performance and overhead.

6.1 Experimental Setup

The experiments are performed on a virtual machine on a host with four Intel(R) Core(TM) i7-6700 CPU @3.40 GHz with 32 GB RAM. The virtual machine has four processors, 16 GB RAM, and runs Ubuntu 16.04.1. The version of Redis is 3.0.3. The experiments use seventeen application traces from three provenance systems or tools (i.e., PASS, SPADE and audit respectively). The PASS traces include seven normal applications [44] and seven vulnerable applications as shown in Table 2. The datasets are available online (<http://yulaixie.cn/>). The DARPA APT trace includes a wide variety of normal user behavior in APT test and is publicly available [43]. This trace originally consists of a set of files which are the output from a particular Technical Area performer. As each of these files is very big and some of these files contain invalid data, we choose a representative trace file (ta1-trace-e3-official-1.json) that contains valid and useful data. Further, we extract the dependency relationships from the file and map the nodes in each relationship into integer numbers. Then we use K-means algorithm to cluster these relationships into different events. Each event contains not more than 1,000 relationships to avoid too long detection time. We select a part of the events as the training set and the rest as test set to make the false alarm rate as low as possible. In general, these four kinds of application traces represent different workload types which include biological workload, filesystem workload, system compiling workload, and application server workload. They vary in different size and have different number of provenance traces for the training set and test set. For instance, the size of blast-lite is only 95 KB, while the size of NetBSD can be 6 GB.

The normal applications are mainly used to demonstrate whether the intrusion detection system has a low false alarm rate. The data sets for vulnerable applications include both normal and intrusion traces. The first four vulnerable applications [25] in PASS traces are commonly used server workloads that have a vulnerability in their low software versions. Phishing email [37] simulates the typical APT attack by cheating users to click a malicious email link. ImageTragick and Ransomware [45] are typically occurred attack types that have multiple variants in their executions. Tomcat, WordPress and Webmin are server applications that have recently been exploited.

For applications that are vulnerable to remote attacks, we simulate an attacker to initiate an exploit via the Metasploit Framework from a remote machine. After attaining the root shell, the attacker can perform any operations she want. Typical operations include files download, read, write, deletion, etc. These operations generate syscalls that are intercepted by the PASS, SPADE or CamFlow collection components and then converted to the provenance information. For applications that are vulnerable to local attacks, the intruders directly execute attack programs on the local machine, and then perform illegal activities which are captured by the provenance collection components.

To achieve the completeness of the training set, the training set has covered different application behavior. For instance, the training set for vsftpd includes files upload, download, read, write, and deletion. We have also tried

TABLE 2
Basic Descriptions for a Variety of Normal and Vulnerable Applications Used in Intrusion Detections

Application	Description	# of traces	Training set			Test set		Provenance system
			# of traces	# of nodes	# of dependencies	# of normal traces	# of intrusion traces	
blast-lite	A simple instance of the Blast biological workload	1	1	350	628	1	0	PASS
postmark	The PostMark file system benchmark	1	1	7818	4777	1	0	PASS
patch-apr17	Patching the Linux kernel	2	1	64645	33919	1	0	PASS
chall3	Image processing using workflow	2	1	14707	11273	1	0	PASS
cp-lite	copy a file	1	1	55	119	1	0	PASS
elaine-oct25	A researcher developed a python application and wrote a conference paper	47	40	117069	292071	7	0	PASS
NetBSD	Build of several components of NetBSD	616	300	5254454	8762678	50	0	PASS
vsftp (CVE-2011-2523)	A secure FTP server	510	50	19515	15213	350	110	PASS
samba (CVE-2007-2447)	A program that implements the SMB (Server Messages Block) protocol and provides file sharing services	210	10	1897	7367	90	110	PASS
distcc (CVE-2004-2687)	A distributed, C++ compiler tool	333	19	1468	1142	204	110	PASS
proftpd (N/A)	Highly configurable GPL-licensed FTP server software	162	80	11776	13222	60	22	PASS
phishing e-mail (CVE-2002-1337)	A user clicks a web page link in an email to open the firefox browser that downloads a malicious backdoor program. The user wrongly executes it that automatically sends sensitive data outside	227	50	64379	70522	77	100	PASS
ImageTragick (CVE-2016-3714 3718))	An attacker uploads a malicious image file which is executed and generates a remote shell in the attacker's host	120	50	109505	112873	20	50	PASS
Ransomware Attack (N/A)	The attacker exploits the vulnerability of the Redis database and gets the root shell. Then it encrypts the database and leaves a note in the home directory of the user. The user can access the database only after it has paid the bitcoins	134	47	23045	19755	37	50	PASS
Tomcat (CVE-2017-12617)	Tomcat programs in versions 8.0.0.RC1 to 8.0.46 can upload JSP files to the server through a special request package. The code contained in the files will be executed by the server. An attacker can exploit this vulnerability to execute malicious code to gain privileges.	201	100	72764	66811	51	50	SPADE
WordPress (CVE-2019-8943)	WordPress programs prior to version 4.9.9 allow path traversal to find available path to write. An attacker can upload a special image containing PHP malicious code. By executing the code, the attacker can obtain the control of the intruded host.	200	100	76125	67170	50	50	SPADE
Samba (CVE-2017-7494)	The intruder exploits the vulnerability of Samba in its version 4.4.13 and attains the root shell.	300	100	3255	6397	100	100	CamFlow
Webmin (CVE-2019-9624)	The Webmin has a vulnerability in its version 1.900. The attacker exploits the vulnerability and attains the root shell. Then she makes a large number of illegal operations.	258	100	5691	15506	94	64	CamFlow
DARPA APT trace	A wide variety of normal user behavior in APT test in the DARPA Transparent Computing (TC) program [43]. These behaviors are from a particular TA1 performer during the Engagement #3 activity period.	287	19	1191481	1498713	268	0	audit

different number of training traces to test the false alarm rate. The training is stopped only when the false alarm rate is as low as possible. For the DARPA APT trace, we also put provenance dependency relationships and their duplicates

in training set and test set respectively in order to achieve a low false alarm rate. Table 2 also shows the number of nodes and dependency relationships in the training set. The intrusion traces in the test set include two types. One is the

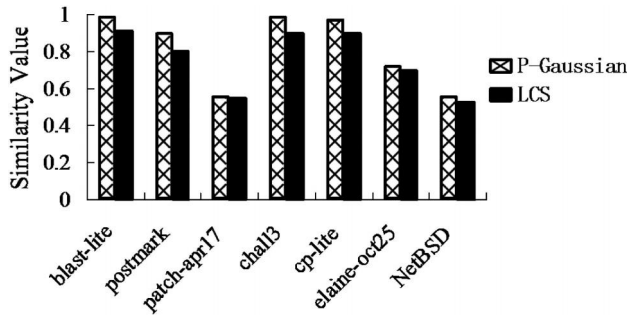


Fig. 5. Similarity value for all normal applications. The similarity values for elaine-oct25 and NetBSD are the average values across all the elaine-oct25 or NetBSD traces.

real attack exploits and payloads. The other is the transformed attacks. These operations include two aspects: (1) reverse the order of operations; (2) add redundant operations to the original intrusion path.

6.2 Similarity Value

The similarity value indicates whether an intrusion has happened by comparing the sequences between the training set and test set. We first measure the similarity value for all the PASS normal applications for P-Gaussian and LCS [13] as shown in Fig. 5. A large similarity value will be more likely to avoid false alarms. The similarity values are large (0.799–0.986) for blast-lite, postmark and cp-lite since these applications use the same single trace for both training set and test set. The similarity values for chall3 are also large (0.9 for LCS and 0.986 for P-Gaussian) because its test trace is generated using the completely same process and arguments as the training process. The difference just lies in that the process in the test trace does not execute successfully and can be viewed as a variant of the training trace. However, P-Gaussian outperforms LCS by 5.92–12.3 percent on all these traces, indicating that it exploits more common sequences between the trace and itself or its variant. The similarity values for patch-apr17, elaine-oct25 and NetBSD are comparatively small since they compile different files between training traces and test traces. However, similarly, P-Gaussian still achieves comparable or better performance on these data sets.

6.3 False Alarm Rate and Detection Rate

To further evaluate the effectiveness of P-Gaussian, we compare it with FRAP [26], Pagoda [23], PIDAS [25], and LCS [13] on ten applications from three collection systems as shown in Table 2. The first three methods are all provenance based intrusion detection methods developed in recent years. For fairness, LCS also uses the provenance graph based similarity calculation method after computing the sequence similarity for each path. For P-Gaussian and LCS, the threshold T is set to judge whether the provenance graph of user behavior is similar to the intrusion behavior profile in the rule database. For PIDAS, the threshold T is set to judge whether a certain length of provenance path is suspicious enough to be generated by intrusion behavior. The length is three as it is the recommended result [25]. For Pagoda, it judges intrusion by considering both the anomaly degree of a single path and the whole provenance graph. The alarm is sounded when either of the anomaly degree exceeds the predefined threshold. For FRAP, it computes the feature vector of the provenance records in a sliding window and identifies the intrusion instance outside of the existing clusters using the K-means clustering algorithm. The detection rate shows the percentage of intrusion behaviors that are correctly classified. The false alarm rate reveals the percentage of normal behaviors that are reported as intrusions.

1) PASS trace. We evaluate the detection performance by plotting the average and median results of all the intrusion data sets using ROC curve as shown in Fig. 6. Each point in the figure shows the performance of a method at a certain threshold. The best results are achieved by the points in the top-left corner. FRAP has only one point in the figure as it does not define multiple thresholds but judges whether an intrusion instance is located in the existing clusters in the rule database. As P-Gaussian is significantly impacted by the similarity between an input provenance graph and the rule database, Fig. 6 mainly shows the points with a proper similarity value that may be practical useful.

P-Gaussian performs the best in total. Specifically, it has a lower false alarm rate and a higher detection rate in the median case. This indicates that the average performance has been impacted by the worst numbers and the majority of the cases show a very good performance in practical.

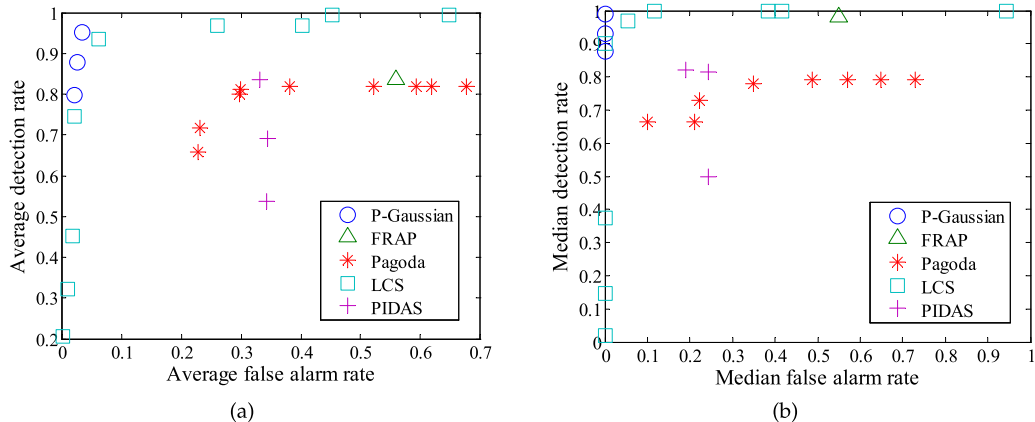


Fig. 6. Composite result for all the intrusion methods on all provenance intrusion data sets in PASS traces. The point located in the upper left corner has a higher detection rate and a lower false alarm rate, and thus shows a better performance. P-Gaussian performs the best in all the intrusion detection algorithms.

TABLE 3
Breakdown of PASS Vulnerable Traces for Different Detection Methods

Application	Average false alarm rate					Average detection rate				
	P-Gaussian	FRAP	Pagoda	LCS	PIDAS	P-Gaussian	FRAP	Pagoda	LCS	PIDAS
vsftp	0.061	0.1	0.99	0.056	0.055	0.85	0.99	0.664	0.645	0.393
samba	0.097	0.98	0.506	0.084	0.062	0.89	1	0.664	0.718	0.646
distcc	0.097	0.006	0.132	0.122	0.062	0.746	0	0.664	0.561	0.433
proftp	0.18	0.52	0.57	0.18	0.53	0.95	0.91	1	0.90	1
phishing email	0.34	0.974	0.222	0.293	0.909	0.80	0.97	0.603	0.737	0.5
ImageTragick	0.306	0.55	0.322	0.356	0.533	0.647	1	0.922	0.624	0.907
Ransomware Attack	0.345	0.784	0.249	0.369	0.225	0.953	0.98	0.993	0.933	0.94

LCS has a vast distribution in both detection rate and false alarm rate. It does not perform well in some cases since it has lower detection accuracy than P-Gaussian on identifying the intrusion behavior variants with the sequence length increase.

Both PIDAS and Pagoda have a worse performance than P-Gaussian. For PIDAS, the increase in sequence length in the variants makes some of the provenance paths not suspicious any more, resulting in a decrease in detection rate. Yet, PIDAS uses only one provenance path but not a whole provenance graph to judge the program behavior, thus the false alarm rate is higher than P-Gaussian. Though Pagoda identifies intrusion by considering both single provenance path and the whole provenance graph, it cannot detect the variants with the sequence length increase. For FRAP, it does not achieve good performance on false alarm rate. The typical reason is that it only uses a part of provenance records (i.e., provenance in a sliding window) that cannot represent the whole provenance graph feature.

We further investigate the performance results of individual datasets as shown in Table 3. P-Gaussian achieves the best tradeoff between detection rate and false alarm rate in most of the applications. The average false alarm rates of P-Gaussian for phishing email, ImageTragick and Ransomware are a little high. This is because the normal sequence may have some common items with the anomaly sequence in the rule database. When the threshold is small, the provenance graph that contains a large number of such normal sequences may easily be judged as anomalous. The average

detection rates of P-Gaussian for distcc and ImageTragick are not high. The main reason is that the intruders have performed a large number of normal behaviors to interfere the detection. This makes the anomaly degrees of both the provenance path and the whole provenance graph are low. Thus the provenance graph that describes the intrusion behavior will have a low similarity value to the rule database. Thus it cannot be accurately identified when the threshold is large.

2) SPADE trace. Fig. 7 shows ROC curves of two SPADE application traces for different intrusion detection methods. P-Gaussian performs the best in the most of the case. FRAP has a low detection rate as it only considers part of the event graph as detection object. Pagoda has a vast distribution in the false alarm rate, especially the false alarm rate is very big when the graph threshold is small. Similarly, P-Gaussian achieves a low detection rate when the threshold is small, which makes two provenance sequences more easily be judged as similar. This indicates that we can choose a proper threshold to achieve the best result.

3) CamFlow trace. Fig. 8 shows ROC curves of two camflow application traces for different intrusion detection methods. P-Gaussian also performs the best in the most of the case. FRAP performs not good as it only considers part of the provenance graph in a sliding window as detection object. Pagoda has a high false alarm rate, even when the graph threshold is small. Similar to SPADE trace, P-Gaussian achieves a low detection rate when the threshold is small. We can choose a proper threshold to achieve the best result.

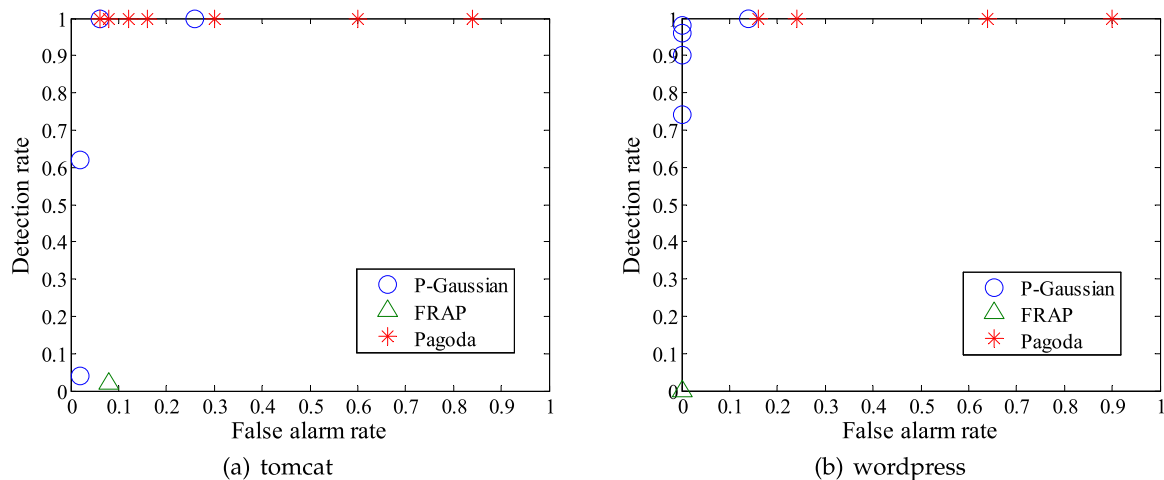


Fig. 7. ROC curves of different SPADE application traces for different intrusion detection methods. The point located in the upper left corner has a higher detection rate and a lower false alarm rate, and thus shows a better performance.

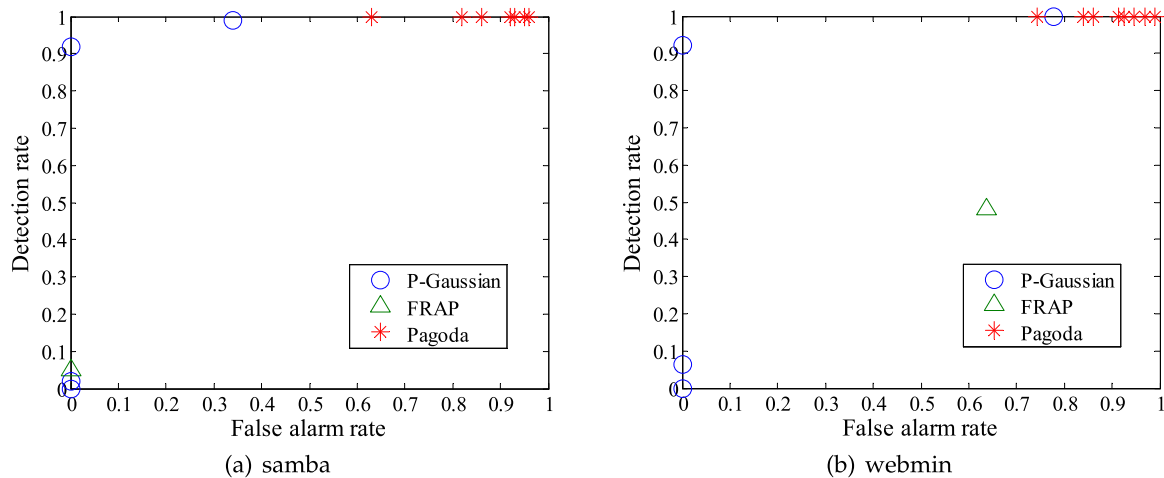


Fig. 8. ROC curves of different camflow application traces for different intrusion detection methods. The point located in the upper left corner has a higher detection rate and a lower false alarm rate, and thus shows a better performance.

4) DARPA APT trace. As we put provenance information and its duplicates in training set and test set respectively, the provenance path in the test set can be also found in the rule database. Thus the false alarm rates of P-Gaussian and Pagoda that depend on path comparison are both 0. The false alarm rate of FRAP is 165/268 when the iteration times of generating a feature vector from the provenance graph is four. This false alarm rate can only be decreased to 114/268 when the iteration times is 20 and is hard to be decreased any more. The main reason is that the original training set can form a number of clusters, each of which contains a bunch of similar vectors. Each test instance can also generate a set of vectors that form a cluster. However, the Euler distance between the cluster for test instance and the clusters in the rule database can be beyond the radius of the clusters in the rule database. In this case, the test instance can be wrongly regarded as an anomaly.

6.4 Parameters and Threshold Chosen

For the similarity function $F(m)$ (i.e., Equation (2)), we convert it to normal Gaussian distribution, thus $\mu = 0$, $\sigma = 1$. We also investigate how the threshold impacts the experimental results as shown in Fig. 9. The average detection rate and false alarm rate both decrease with the increase of the threshold. This is because it is more difficult and strict to identify an intrusion when the threshold

for similarity calculation has increased. Comprehensively, P-Gaussian gets the best result when the threshold is 0.5 by considering the tradeoff between both detection rate and false alarm rate.

We further see the performance results as shown in Table 4. We think the result is good when the detection rate is above 0.9 and false alarm rate is not more than 0.1. Most of the applications achieve the best result when threshold is still 0.5. Distcc achieves the best result when the threshold is 0.2. This is because distcc has generated different operations in the normal behavior, resulting a low similarity to the rule database. Thus the false alarm rate is low even when the threshold is small. Comparatively, samba achieves the best result when the threshold is 0.7. This is mainly due to a large number of sequences that are similar to the sequences in the rule database generated in the intrusion behavior, resulting in a high detection rate even when the threshold is large.

6.5 Detection Time and Throughput

Fig. 10 shows the detection time and throughput of P-Gaussian in the case of multiple threads and multiple instances for ransomware. Each number was got by running the test six times and computing the average number. Overall, P-Gaussian has got a better performance with the increase of multiple threads and multiple instances. However, the detection performance does not increase by double as the number of threads double. This is because the whole detection process has been divided into three parts: looking for head nodes, computing the similarity value between the paths that start from the head nodes and the paths in the rule database, and computing the similarity value of the whole provenance graph. Only the paths comparison using Gaussian distribution has been speeded up using multiple threads or instances. We have also tried to parallelize the processes of both looking for head nodes and provenance graph similarity calculation. However, we have to use locks to avoid resource contention which may decrease the performance. We also see that the multiple instances do not bring significant performance improvement in the case of the same multiple threads. A possible reason is that the similarity calculation of provenance graph needs all the

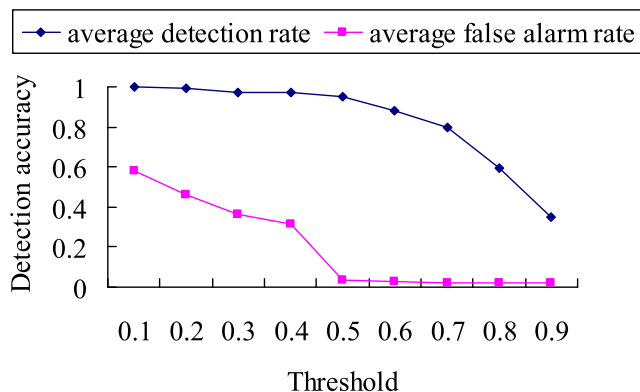


Fig. 9. Average detection rate and false alarm rate on all the PASS vulnerable applications with different thresholds.

TABLE 4
Breakdown of PASS Vulnerable Traces for Different Thresholds

Application	False alarm rate									Detection rate								
	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
vsftp	0.075	0.075	0.07	0.065	0.055	0.055	0.055	0.05	0.05	1	1	1	1	1	1	0.88	0.49	0.29
samba	0.11	0.11	0.11	0.11	0.1	0.1	0.078	0.078	0.078	1	1	1	1	1	1	1	0.82	0.18
distcc	0.46	0.1	0.086	0.086	0.086	0.028	0.014	0.014	0	0.99	0.96	0.81	0.8	0.8	0.73	0.72	0.68	0.23
proftp	0.45	0.42	0.38	0.38	0	0	0	0	0	1	1	1	1	0.91	0.91	0.91	0.91	0.91
phishing email	0.95	0.90	0.77	0.44	0	0	0	0	0	1	1	1	1	0.99	0.93	0.87	0.41	0.01
ImageTragick	1	0.8	0.5	0.45	0	0	0	0	0	1	1	1	1	0.96	0.58	0.22	0.04	0.02
Ransomware Attack	1	0.81	0.65	0.65	0	0	0	0	0	1	1	1	1	1	1	0.98	0.8	0.8

subtotal results of paths on the individual instance. In general, the throughput is not very high because of the frequent sequence similarity comparison. This can be further improved through adopting the specialized hardware such as GPU.

6.6 Query Time

We mainly make forensic analysis via initiating the following three kinds of queries:

- Q.1) Given a detection point such as a damaged file, find all the ancestors on which it directly depends.
- Q.2) Given a detection point such as a damaged file, find all the ancestries that contain the intrusion sources and the intermediate nodes along the intrusion path.
- Q.3) Search for all the files that an intrusion source has accessed.

These queries cover basic cases in forensic analysis and represent different query complexity. The first two queries are backward queries that search for the parents or all the ancestors of a given file respectively. The third query is a forward query that involves the retrieval of all the files that may have been damaged or leaked. To make the test fair, we reboot machine before each test.

Table 5 shows the query performance for different intrusion detection methods on the vsftp application. Each number in the table is the average value of at least five trials. P-Gaussian performs the best because it classifies the hot and cold provenance, and almost all the hot provenance is stored in Redis. So the queries in three cases only get data from memory. Pagoda can get a similar performance as P-Gaussian when provenance size is small. When

provenance is too big to put into Redis, part of the provenance will be streamed into disks. As Pagoda does not support classification of hot and cold provenance, even the hot provenance to be queried again may be found on disks. So this can have a big impact on the Pagoda query performance. FRAP gets original provenance from disks and then performs detection process in memory using GraphChi. To speed up its query process, we first store the provenance in the BerkeleyDB databases, and then perform queries on them. However, the query performance is still worse than P-Gaussian. The main reason is the time-consuming disk IO when getting data from BerkeleyDB. In addition, the provenance pruning in P-Gaussian further improves its performance. We also see that Pagoda performs worse than FRAP in the third query. This is due to the recursive queries between provenance nodes in Redis and BerkeleyDB.

6.7 Overhead Analysis

Memory Overhead. Table 6 shows the memory overhead for P-Gaussian on different intrusion applications. The memory overhead mainly consists of two parts: memory used by the Redis database to store provenance and the memory consumed by the detection process. The memory overhead is about 76 MB–164 MB. This takes up only a small part of a 16 GB system memory.

CPU Overhead. Fig. 11 shows the CPU utilization for P-Gaussian and Pagoda on 100 DARPA APT normal events.

TABLE 5
Query Time (us) for Different Intrusion
Detection Methods on vsftp

Categories	P-Gaussian	Pagoda	FRAP
Q.1	113	1,105	31,269
Q.2	3,801	4,162	124,532
Q.3	46,037	106,993	85,697

TABLE 6
Memory Overhead (MB) of P-Gaussian
for Different Applications

Applications	Memory overhead		
	Redis	Detection process	Total
Imagetracik	24	96	120
Phishing email	16	148	164
Proftp	8	120	128
Ransomware	8	68	76

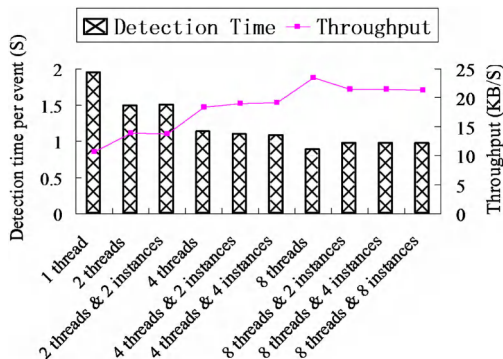


Fig. 10. Detection time and throughput in the case of multiple threads and multiple Redis instances.

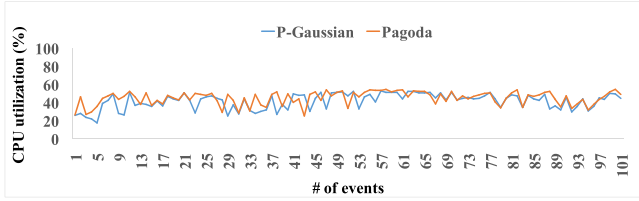


Fig. 11. CPU utilization for different intrusion detection methods.

P-Gaussian consumes a comparable CPU utilization with Pagoda. This is because they both traverse all the paths in the provenance graph and then compute the anomaly degree of the provenance graph by considering the anomaly degree of each path. However, the CPU overhead is acceptable (around 40 percent).

Runtime Overhead. The provenance collection for intrusion detection can have impact on the application performance. In this test, the experimental machine provides Apache network service. We utilize web application stress tool at the client side to send read or write requests to the server to simulate the intrusion attack at different workloads. The tool simulates 100 clients to access the HTML files in three different sizes (5 KB, 20 KB, 50 KB) at the Apache server. The request intervals are set to be 0, 3–5 seconds, 60–90 seconds to simulate the workload from heavy to light. The performance overhead is minimal (0.132–0.264 percent) in the latter two cases. Even in the heaviest workload, the overhead is maximumly only 10.76 percent.

Storage Overhead. 1) Provenance growth overhead. We investigate the provenance increase overhead as shown in Fig. 12. The original provenance is generated using PASS framework without any pruning. Pagoda reduces provenance size by filtering the noise and aggregating the similar provenance items. P-Gaussian further compresses provenance by classifying hot and cold provenance and compresses the cold provenance using bzip2. Fig. 12 shows the best case of P-Gaussian when all the provenance is cold. In practical, part of provenance is hot and stored in Redis. However, P-Gaussian still achieves better provenance space utilization than Pagoda. 2) Size of rule database. We make statistical analysis of the size of rule database for seven vulnerable applications in PASS trace. The size of rule database is from 16.4 KB to 352.3 KB for P-Gaussian. This is acceptable for today's memory and disk.

7 DISCUSSION

When the intrusion behavior is induced by the malware (typically virus or worm), we do not consider some complex cases, such as virus encryption, packaging, dynamic deformation, and virus splitting. However, P-Gaussian can still be applied in these cases. For instance, we can first decrypt a virus, and then apply P-Gaussian to identify the virus variant in the case of sequence length increase or sequence order transformation.

P-Gaussian, not only detects the intrusion behavior variant, but also provides a basis for detecting the zero-day attack. Typically, we can collect the provenance graphs that describe both the normal behavior and intrusion behavior using P-Gaussian. Then we can compute the propagation probability of the provenance object instance using

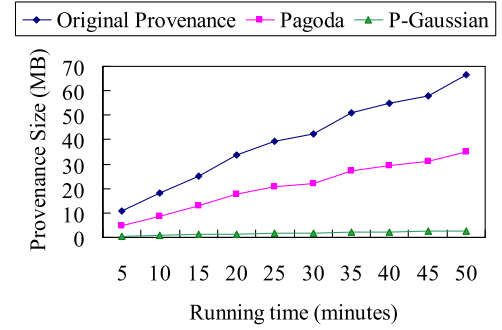


Fig. 12. Provenance storage overhead during kernel compilation.

Bayesian-network [46] to accurately identify the zero-day attack path.

Modern attacks such as Advanced Persistent Threat (i.e., APT) are stealthy in a long time and usually has a long attack chain. Thus it usually involves analyzing and processing a large amount of provenance data. Existing researches prune provenance graphs by partitioning the execution units [47], [48], classifying logging and tainting [37], or compressing provenance graphs by employing web compressing techniques [49] and thus simplify the analysis. These approaches can be used on top of the provenance optimization method proposed in this paper to support the detection of APT attack.

Except the algorithm optimization, some existing researches adopt GPU [22] to achieve the goal of real-time detection. This brings in a negligible hardware cost. With the rise and population of memory database in recent years, it is efficient to leverage these memory databases to speed up intrusion detection and forensic analysis especially in big data environments. It is critical to analyze the characteristics of the data itself and choose a proper database. For instance, in a provenance graph, many provenance nodes can share a common ancestor. When we store the ancestor as key and each provenance node as value in a key-value memory database such as Redis, we can aggregate these values into a set and use only one provenance record to store these information in Redis. As Redis supports multiple instances, we also bind each instance to a thread to speed up the sequence similarity comparison in detection process.

The limitation of this work lies in that some of the intrusions or their variants cannot be tracked as they do not go through the syscall interface and produce provenance. In addition, though we prune provenance for highly efficient intrusion detection, the intrusion information may still be conveyed via the pruned files.

Strictly speaking, as variant detection is a very complicated problem, the current method may only be an important and primary approach that steps to the final solution. For instance, the proposed technique only considers the sequence order and length transformation, but for the part in the variant that is not similar to the original sequence, the proposed technique does not further explore the relationship between the part and the original sequence. Typically, the intrusion behavior of the variant usually has inherent relationship with the original intrusion behavior. Exploring this relationship can further enhance and increase the detection accuracy. Therefore, though we have validated the effectiveness of P-Gaussian, it is possible to achieve a more accurate model to describe the variant sequence detection

through considering more factors. Except that, we may need make more accurate evaluation of the mathematics curves from the characteristics of the curves, e.g., distribution mass concentration, tail end behavior, etc.

8 RELATED WORK

Intrusion Behavior Variant Detection. There are generally two categories of methods to detect the variants in host-based intrusion detection system: static analysis and dynamic analysis. The former focuses on using the static signatures (such as the frequently occurring bytes [4] and binary analysis [3]) in the code without executing them to identify the malware variants. But it is inefficient to detect the unknown malware variant [5]. The latter concentrates on the runtime behavior, typically using the control flow based graph [6], [7], [8], [9] to describe the dynamical intrusion behavior. However, these methods either run in a restricted environment [10] or are usually complicated as they need to analyze the complete intrusion behavior in the form of the whole run time graph, consuming much time and space overhead. There is also a combination of dynamic and static analysis, such as malicious browser extension [50] and malware variant analysis in Android [51]. The recent work computes the similarity of the sequences for transformed data leak [52]. This paper is more general to be applied to all kinds of transformation and intrusion behavior variants.

Sequence Comparison in Intrusion Detection. There have been some existing works on sequence comparison in intrusion detection. Typically, the traditional system-call based intrusion detection methods compute the similarity between a fixed length of the system call sequence and the sequences in the rule database. For instance, Hofmeyr *et al.* [12] propose to use Hamming distance to calculate the number of common system calls between the detected system calls and the closest match in the rule database. Warrender *et al.* [53] further use a locality frame to count how many mismatches have happened, once it exceeds a predefined threshold, the program is judged to be abnormal. Rare sequences can also be considered mismatches in this case. There are also other methods (e.g., data mining [54], statistical analysis [55] or finite state machine [56]) to exploit the system call sequences to improve the detection efficiency. The recent work [57] also analyzes the system call behavior in the malware execution, the probability of system call transition represented as a matrix [36], and the correlation of the events in the long-term program behavior [58]. However, how to detect the transformed intrusion behavior sequence when sequence length or order has changed has not been studied in these methods. There are also some classical algorithms (such as LCS [13] and GST [14]) to compare two different sequences. However, these algorithms are greatly impacted by the sequence length or order transformation.

Provenance-Based Intrusion Detection. Provenance-based detection method has recently been used on PaaS cloud [26] and IoT device [59]. They both use a provenance graph to model the application behavior. Lemay *et al.* [60] also design a series of rule-based grammars to identify the anomaly in the provenance data. In previous work, we have developed PIDAS [25] that uses data provenance to both detect

intrusion and make forensic analysis. Further, we design Pagoda [23] to identify intrusions in big data environments and employ a memory database to store provenance data and detect intrusions in real time. These methods do not aim for detecting the variant in the case of the length of the provenance sequence increases.

Gaussian Distribution in Intrusion Detection. Gaussian distribution has been used in intrusion detection to filter network traffic data [18] or provide deployment strategy in wireless sensor networks [19], [20], while this paper aims to detect the variants by computing the similarity between different sequences in host-based intrusion detection systems.

Intrusion Detection Using Multi-Thread. Numerous network intrusion detection systems (NIDS) [22], [61], [62], [63], [64], [65], [66] use multiple CPU or GPU to process network packets in multi-thread in order to achieve high throughput and real-time performance. This paper focuses on host-based behavior detection and analysis using multiple Redis instances with multi-thread.

Provenance-Based Forensic Analysis. Provenance or causality-based context can accurately describe dependencies between intrusion process and the infected objects, and have been broadly used to analyze how intrusion has happened [33], [37], [45], [47], [48], [67], [68], [69], [70], [71]. For instance, Pohly *et al.* [33] propose to collect the high fidelity provenance to accurately analyze the intrusion. Jiang *et al.* [68] focus on the worm infection by using colors to differentiate a wide variety of server processes and thus reduce the log data to analyze. Goel *et al.* [70] use provenance as a tool to help the system rebuild after the intrusion inflicts damage to the system. Both Lee *et al.* [47] and Ma *et al.* [37], [71] prune provenance or audit logs to simplify and speed up the forensic analysis. In recent, Pasquier *et al.* [41] develop a Linux Security Module that is incorporated into existing operating systems to provide realtime provenance runtime analysis for typical security applications. Our method differs in that we classify hot and cold provenance to speedup provenance query in forensic analysis and save storage space simultaneously.

9 CONCLUSION

Detecting intrusion behavior variants has long been a great challenge. We have presented P-Gaussian, a novel provenance-based Gaussian distribution scheme to accurately describe and identify intrusion behavior variants when their sequence length increases or sequence order transforms. We have further developed a prototype that achieves high efficient and real-time detection performance on big provenance streams by exploiting the parallelism of provenance processing using multiple Redis instances and multi-threading technology in multi-core environments. We also classify hot and cold provenance to accelerate forensic analysis and reduce provenance storage overhead simultaneously. Experimental results have proven the performance and efficiency of our system and method.

ACKNOWLEDGMENTS

The author would like to thank the anonymous reviewers for their constructive and detailed comments. This work was supported in part by the National Science Foundation

of China under Grant No. 61972449, U1705261 and 61821003, CCF-NSFOCUS KunPeng research fund, Wuhan Application Basic Research Program under Grant No. 2017010201010104, Hubei Natural Science and Technology Foundation under Grant No. 2017CFB304, and the Fundamental Research Funds for the Central Universities under Grant No. 2019kfyXKJC021.

REFERENCES

- [1] Zeus variants, 2013. [Online]. Available: <http://www.tech-faq.com/the-zeus-virus.html>
- [2] New variant of WannaCry, 2017. [Online]. Available: <http://www.zerohedge.com/news/2017-05-15/second-wave-ransomware-cyberattack-begins-spread-wever-never-seen-anything>
- [3] D. Song *et al.*, "BitBlaze: A new approach to computer security via binary analysis," in *Proc. Int. Commission Intervention State Sovereignty*, 2008, pp. 1–25.
- [4] S. Yu, S. Zhou, L. Liu, and R. Yang, "Detecting malware variants by byte frequency," *J. Netw.*, vol. 6, no. 4, pp. 638–645, 2011.
- [5] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proc. Annu. Comput. Security Appl. Conf.*, 2007, pp. 421–430.
- [6] H. Yin, D. Song, M. Egele, C. Kruegel, and E. Kirda, "Panorama: Capturing system-wide information flow for malware detection and analysis," in *Proc. ACM Conf. Comput. Commun. Security*, 2007, pp. 116–127.
- [7] D. Shin, K. Lee, and D. Won, "Malware variant detection and classification using control flow graph," in *Proc. Int. Conf. Hybrid Inf. Technol.*, 2011, pp. 174–181.
- [8] S. Cesare, Y. Xiang, and W. Zhou, "Control flow-based malware variant detection," *IEEE Trans. Dependable Secure Comput.*, vol. 11, no. 4, pp. 304–317, Jul./Aug. 2014.
- [9] G. Liang, J. Pang, and C. Dai, "A behavior-based malware variant classification technique," *Int. J. Inf. Educ. Technol.*, vol. 6, no. 4, pp. 291–295, 2016.
- [10] K. Rieck, T. Holz, C. Willems, P. Ssel, and P. Laskov, "Learning and classification of malware behavior," in *Proc. Conf. Detection Intrusions Malware Vulnerability Assessment*, 2008, pp. 108–125.
- [11] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for unix processes," in *Proc. IEEE Symp. Security Privacy*, 1996, pp. 120–128.
- [12] S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *J. Comput. Security*, vol. 6, no. 3, pp. 151–180, 1998.
- [13] G. S. Lueker, "Improved bounds on the average length of longest common subsequences," *J. ACM*, vol. 56, no. 3, pp. 17:1–17:38, 2009.
- [14] M. J. Wise, "String similarity via greedy string tiling and running karp-rabin matching," Ph.D. dissertation, Dept. Comput. Sci., Univ. Sydney, Camperdown NSW, 1993.
- [15] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989.
- [16] S. Shah, C. A. N. Soules, G. R. Ganger, and B. D. Noble, "Using provenance to aid in personal file search," in *Proc. USENIX Annu. Tech. Conf.*, 2007, pp. 171–184.
- [17] S. T. King and P. M. Chen, "Backtracking intrusions," *ACM Trans. Comput. Syst.*, vol. 23, no. 1, pp. 51–76, 2005.
- [18] Y. Wang, S. Xu, W. Liu, and Q. Huang, "A Gaussian distribution-based lightweight intrusion detection model," *Int. J. Comput. Sci. Netw. Security*, vol. 15, no. 9, pp. 6–11, 2015.
- [19] Y. Wang and Z. Lun, "Intrusion detection in a K-Gaussian distributed wireless sensor network," *J. Parallel Distrib. Comput.*, vol. 71, pp. 1598–1607, 2011.
- [20] Y. Wang, W. Fu, and D. P. Agrawal, "Gaussian versus uniform distribution for intrusion detection in wireless sensor networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 2, pp. 342–355, Feb. 2013.
- [21] E. Lukacs and E. P. King, "A property of normal distribution," *The Ann. Math. Statist.*, vol. 25, no. 2, pp. 389–394, 1954.
- [22] M. Jamshedi *et al.*, "Kargus: A highly scalable software-based intrusion detection system," in *Proc. ACM Conf. Comput. Commun. Security*, 2012, pp. 317–328.
- [23] Y. Xie, D. Feng, Y. Hu, Y. Li, S. Sample, and D. Long, "Pagoda: A hybrid approach to enable efficient real-time provenance based intrusion detection in big data environments," *IEEE Trans. Dependable Secure Comput.*, to be published, doi: 10.1109/TDSC.2018.2867595.
- [24] Bzip2 compression algorithm, 2018. [Online]. Available: <http://www.bzip.org/>
- [25] Y. Xie, D. Feng, Z. Tan, and J. Zhou, "Unifying intrusion detection and forensic analysis via provenance awareness," *Future Gener. Comput. Syst.*, vol. 61, pp. 26–36, 2016.
- [26] X. Han, T. Pasquier, T. Ranjan, M. Goldstein, and M. Seltzer, "FRAPPuccino: Fault-detection through runtime analysis of provenance," in *Proc. USENIX Workshop Hot Topics Cloud Comput.*, 2017, pp. 1–7.
- [27] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. I. Seltzer, "Provenance-aware storage systems," in *Proc. USENIX Annu. Tech. Conf.*, 2006, pp. 1–14.
- [28] K.-K. Muniswamy-Reddy *et al.*, "Layering in provenance systems," in *Proc. USENIX Annu. Tech. Conf.*, 2009, pp. 1–14.
- [29] A. Gehani and D. Tariq, "SPADE: Support for provenance auditing in distributed environments," in *Proc. ACM/IFIP/USENIX Int. Middleware Conf.*, 2012, pp. 101–120.
- [30] R. Spillane, R. Sears, C. Yalamanchili, S. Gaikwad, M. Chinni, and E. Zadok, "Story book: An efficient extensible provenance framework," in *Proc. TaPP'09, First Workshop on the Theory and Practice of Provenance*, 2009, pp. 1–10.
- [31] R. Bose and J. Frew, "Composing lineage metadata with XML for custom satellite-derived data products," in *Proc. Int. Conf. Scientific Statist. Database Manage.*, 2004, pp. 275–284.
- [32] A. Vahdat and T. Anderson, "Transparent result caching," in *Proc. USENIX Annu. Tech. Conf.*, 1998, pp. 25–37.
- [33] D. J. Pohly, S. McLaughlin, P. McDaniel, and K. Butler, "Hi-Fi: Collecting high-fidelity whole-system provenance," in *Proc. Annu. Comput. Security Appl. Conf.*, 2012, pp. 259–268.
- [34] D. Devecsery, M. Chow, X. Dou, J. Flinn, and P. M. Chen, "Eidetic systems," in *Proc. USENIX Symp. Operating Syst. Design Implementation*, 2014, pp. 525–540.
- [35] T. Pasquier *et al.*, "Practical whole-system provenance capture," in *Proc. SoCC '17 Symp. Cloud Comput.*, 2017, pp. 405–418.
- [36] Z. Xu *et al.*, "High fidelity data reduction for big data security dependency analyses," in *Proc. ACM Conf. Comput. Commun. Security*, 2016, pp. 504–516.
- [37] S. Ma, X. Zhang, and D. Xu, "ProTracer: Towards practical provenance tracing by alternating between logging and tainting," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2016, pp. 1–15.
- [38] A. Bates, D. Tian, K. Butler, and T. Moyer, "Trustworthy whole-system provenance for the linux kernel," in *Proc. USENIX Security*, 2015, pp. 319–334.
- [39] J. Lyle and A. Martin, "Trusted computing and provenance: Better together," in *Proc. USENIX Workshop Theory Practice Provenance*, 2010, pp. 1–10.
- [40] R. Hasan, R. Sion, and M. Winslett, "The case of the fake picasso: Preventing history forgery with secure provenance," in *Proc. USENIX Conf. File Storage Technol.*, 2009, vol. 9, pp. 1–14.
- [41] T. Pasquier *et al.*, "Runtime analysis of whole-system provenance," in *Proc. ACM Int. Conf. Comput. Commun. Security*, 2018, pp. 1601–1616.
- [42] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Wenkatakrisnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. 40th IEEE Symp. Security Privacy*, 2019, pp. 1137–1152.
- [43] Transparent computing engagement 3 data release, 2018. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [44] Y. Xie, K.-K. Muniswamy-Reddy, D. Feng, Y. Li, and D. D. E. Long, "Evaluation of a hybrid approach for efficient provenance storage," *ACM Trans. Storage*, vol. 9, no. 4, pp. 1–29, 2013.
- [45] W. U. Hassan, M. Lemay, N. Aguse, A. Bates, and T. Moyer, "Towards scalable cluster auditing through grammatical inference over provenance graphs," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2018, pp. 1–15.
- [46] X. Sun, J. Dai, P. Liu, A. Singhal, and J. Yen, "Using Bayesian networks for probabilistic identification of zero-day attack paths," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 10, pp. 2506–2521, Oct. 2018.
- [47] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2013, pp. 1–16.
- [48] S. Ma, J. Zhai, F. Wang, K. H. Lee, X. Zhang, and D. Xu, "MPI: Multiple perspective attack investigation with semantics aware execution partitioning," in *Proc. USENIX Security Symp.*, 2017, pp. 1111–1128.

- [49] Y. Xie, K.-K. Muniswamy-Reddy, and L. Chen, "A hybrid approach for efficient provenance storage," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2012, pp. 1752–1756.
- [50] N. Jagpal *et al.*, "Trends and lessons from three years fighting malicious extensions," in *Proc. USENIX Conf. Security Symp.*, 2015, pp. 579–593.
- [51] M. Sun, X. Li, J. C. S. Lui, R. T. B. Ma, and Z. Liang, "Monet: A user-oriented behavior-based malware variants detection system for Android," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 5, pp. 1103–1112, May 2017.
- [52] X. Shu, D. Yao, and B. G. Ryder, "A formal framework for program anomaly detection," in *Proc. Int. Symp. Res. Attacks Intrusions Defenses*, 2015, pp. 270–292.
- [53] C. Warrender, S. Forrest, and B. Pearlmutter, "Detecting intrusions using system calls: Alternative data models," in *Proc. IEEE Symp. Security Privacy*, 1999, pp. 133–145.
- [54] W. Lee and S. J. Stolfo, "Data mining approaches for intrusion detection," in *Proc. Conf. USENIX Security Symp.*, Jan. 1998, pp. 79–93.
- [55] D. Wagner and D. Dean, "Intrusion detection via static analysis," in *Proc. IEEE Symp. Security Privacy*, 2001, pp. 156–168.
- [56] E. Sekar, M. Bendre, D. Dhurjati, and P. Bollineni, "A fast automation-based method for detecting anomalous program behaviors," in *Proc. IEEE Symp. Security Privacy*, 2001, pp. 144–155.
- [57] J. Saxe, D. Mentis, and C. Greamo, "Visualization of shared system call sequence relationships in large malware corpora," in *Proc. VizSec '12 Int. Vis. Cyber Security*, 2012, pp. 33–40.
- [58] X. Shu, D. Yao, and N. Ramakrishnan, "Unearthing stealthy program attacks buried in extremely long execution paths," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Security*, 2015, pp. 401–413.
- [59] E. Nwafor, A. Campbell, and G. Bloom, "Anomaly-based intrusion detection of IoT device sensor data using provenance graphs," in *Proc. 1st Int. Workshop Security Privacy Internet-of-Things*, 2018, pp. 1–4.
- [60] M. Lemay, W. U. Hassan, and T. Moyer, "Automated provenance analytics: A regular grammar based approach with applications in security," in *Proc. USENIX Workshop Theory Practice Provenance*, 2017, pp. 12–12.
- [61] V. Pathak and V. S. Ananthanarayana, "A novel multi-threaded k-means clustering approach for intrusion detection," in *Proc. IEEE Int. Conf. Comput. Sci. Autom. Eng.*, 2012, pp. 757–760.
- [62] B. Haagdorens, T. Vermeiren, and M. Goossens, "Improving the performance of signature-based network intrusion detection sensors by multi-threading," in *Proc. 5th Int. Conf. Inf. Security Appl.*, 2004, pp. 188–203.
- [63] G. Vasiladis, M. Polychronakis, and S. Ioannidis, "MIDeA: A multi-parallel intrusion detection architecture," in *Proc. ACM Conf. Comput. Commun. Security*, Oct. 2011, pp. 297–308.
- [64] G. Vasiladis, M. Polychronakis, S. Antonatos, E. P. Markatos, and S. Ioannidis, "Regular expression matching on graphics hardware for intrusion detection," in *Proc. 12th Int. Symp. Recent Advances Intrusion Detection*, 2009, pp. 265–283.
- [65] R. Patil, H. Dudeja, S. Gawade, and C. Modi, "Protocol specific multi-threaded network intrusion detection system (PM-NIDS) for DoS/DDoS attack detection in cloud," in *Proc. 9th Int. Conf. Comput. Commun. Netw. Technol.*, 2018, pp. 1–7.
- [66] X. Shu, J. Zhang, D. D. Yao, and W. Chun Feng, "Fast detection of transformed data leaks," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 3, pp. 528–542, Mar. 2016.
- [67] D. Farmer and W. Venema, "Forensic computer analysis: An introduction," *Dr. Dobb's J.*, vol. 25, no. 9, pp. 70, 72–75, Sep. 2000.
- [68] X. Jiang, A. Walters, F. Buchholz, D. Xu, Y. Wang, and E. H. Spafford, "Provenance-aware tracing of worm break-in and contaminations: A process coloring approach," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, 2006, pp. 38–45.
- [69] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proc. ACM Symp. Operating Syst. Princ.*, 2003, pp. 223–236.
- [70] A. Goel, K. Po, K. Farhadi, Z. Li, and E. D. Lara, "The taser intrusion recovery system," in *Proc. ACM Symp. Operating Syst. Princ.*, 2005, vol. 39, pp. 163–176.
- [71] S. Ma *et al.*, "Kernel-supported cost-effective audit logging for causality tracking," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 241–253.



Yulai Xie received the BE and PhD degrees in computer science from the Huazhong University of Science and Technology (HUST), China, in 2007 and 2013, respectively. He was a visiting scholar with the University of California, Santa Cruz, in 2010 and a visiting scholar with the Chinese University of Hong Kong, in 2015. He is currently an associate professor with the School of Cyber Science and Engineering, HUST, China. His research interests include digital provenance, intrusion detection, network storage, and computer architecture. He is a member of the IEEE and ACM.



Yafeng Wu received the master's degree in computer science from the Huazhong University of Science and Technology, in 2019. He is currently working toward the PhD degree at the Huazhong University of Science and Technology (HUST). His research interests include digital provenance and intrusion detection.



Dan Feng received the BE, ME and PhD degrees in computer science and technology from the Huazhong University of Science and Technology (HUST), China, in 1991, 1994, and 1997, respectively. She is a professor and director of Data Storage System Division, Wuhan National Lab for Optoelectronics. She is also dean of the School of Computer Science and Technology, HUST. Her research interests include computer architecture, massive storage systems, parallel file systems, disk array, and solid state disk. She has more than 100 publications in journals and international conferences, including FAST, USENIX ATC, ICDCS, HPDC, SC, ICS and IPDPS. She is a member of the IEEE and ACM.



Darrell Long received the BS degree in computer science from San Diego State University, and the MS and PhD degree from the University of California, San Diego. He is currently a professor of engineering at the University of California, Santa Cruz. He holds the Kumar Malavalli Endowed chair of Storage Systems Research and is director emeritus of the Storage Systems Research Center. His current research interests in the storage systems area include high performance storage systems, archival storage systems and energy-efficient storage systems. His research also includes computer system reliability, video-on-demand, applied machine learning, mobile computing and cyber security. He is fellow of the IEEE and fellow of the American Association for the Advancement of Science (AAAS).

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.